

The Linux System Call Execution Model: An Insight



In the first article in this two-part series, published in the August 2024 issue of *OSFY*, we discussed the role of the C library in system call execution. We talked of how the C library loads system call arguments into architecture-specific registers, generating syscall interrupt, which switches the mode from the user to the kernel. In this final part we will discuss what happens after this – how the kernel handles and executes the system call request on behalf of the user space application, and sends the return value of the system call value back to the user application.

We have already discussed the role of the C library (glibc) in handling the request of system calls from user applications in the first part of this two-part series of articles. We learnt how glibc sets the system call arguments in the CPU registers and executes syscall instruction, which causes an exception (software interrupt) that transfers the control to an exception handler present in the kernel code. I will now discuss in detail system call execution handling from the perspective of the kernel.

System call table

Once the syscall instruction is executed and exception is generated, an exception handler is triggered. But how does the kernel know which exception handler it should invoke for syscall instruction, and how does the exception handler know which system call handler it should invoke? I will explain this but first let's discuss the system call table.

The Linux kernel contains a table called the system call table, which is represented by `sys_call_table` array. This array is defined in `/arch/x86/kernel/syscall_64.c`. Given below are some of the important implementation details about `sys_call_table`.

```
/arch/x86/entry/syscall_64.c

#define __SYSCALL_64(nr, sym, compat) extern asmlinkage void
sym(void) ;
#include <asm/syscalls_64.h>
#undef __SYSCALL_64

#define __SYSCALL_64(nr, sym, compat) [nr] = sym,

typedef void (*sys_call_ptr_t)(void);

extern void sys_ni_syscall(void);

const sys_call_ptr_t sys_call_table[__NR_syscall_max+1] = {
    /*
     * Smells like a compiler bug -- it doesn't work
     * when the & below is removed.
     */
    [0 ... __NR_syscall_max] = &sys_ni_syscall,
#include <asm/syscalls_64.h>
};
```